

B 3.2.3 Abstraction Python (HL Only)



B3.2.3 Explain the concept of abstraction in OOP. (HL Only)

This section explains abstraction in OOP, emphasizing its significance in creating modular code. It covers the use of abstract classes to define common interfaces for subclasses.

What is Abstraction?

In the world of Object-Oriented Programming (OOP), abstraction is a fundamental concept that focuses on hiding complex implementation details and exposing only the essential information to the user. Think of it like using a television remote. You have buttons for changing channels, adjusting volume, and turning the TV on or off – these essential functions are exposed to you. You don't need to understand the intricate electronics inside the remote or the television to operate it. These complex details are hidden away, abstracted from your direct interaction.

Similarly, in programming, abstraction allows us to create simplified models of real-world entities or complex processes by focusing on their key characteristics and behaviours, without getting bogged down in the underlying complexities.

The Significance of Abstraction for Modular Code

Abstraction plays a crucial role in developing modular code fragments. Modularity refers to the practice of breaking down a large and complex system into smaller, independent, and manageable units or modules. Abstraction facilitates this by:

- **Reducing Complexity:** By hiding internal workings, abstraction makes individual code components easier to understand and work with. Each module presents a clear and concise interface, outlining what it does without revealing how it achieves it.
- **Enhancing Reusability:** Well-abstracted modules are often designed with specific functionalities, making them reusable in different parts of the same program or even in other projects. If a module performs a specific task with a clear interface, it can be easily integrated elsewhere without knowing its internal implementation.

- **Improving Maintainability:** When a module's internal implementation is hidden behind an abstract interface, changes or improvements to that implementation can be made without affecting other parts of the system that rely on the interface. This loose coupling makes the code easier to maintain and evolve.
- **Facilitating Collaboration:** In team-based projects, abstraction allows different developers to work on different modules concurrently. As long as everyone adheres to the agreed-upon abstract interfaces (whether explicit or implicit), the internal implementations of their respective modules can vary independently.

Achieving Abstraction with Abstract Base Classes in Python

Python achieves abstraction primarily by using Abstract Base Classes (ABCs) provided by the `abc` module. While Python doesn't have a specific `interface` keyword like Java, the principles of defining contracts and achieving abstraction remain central. Python also leverages the concept of "duck typing" to achieve a form of implicit interface.

Abstract Base Classes (ABCs):

An Abstract Base Class is a class that cannot be instantiated directly. It defines one or more abstract methods, which are methods that must be implemented by any concrete (non-abstract) subclass. To define an ABC in Python, you use the `abc` module.

- You import `ABC` and `abstractmethod` from the `abc` module.
- You make your class inherit from `ABC` as a metaclass (or by inheriting from `abc.ABC`).
- You decorate abstract methods with `@abstractmethod`.

Example (Python):

```

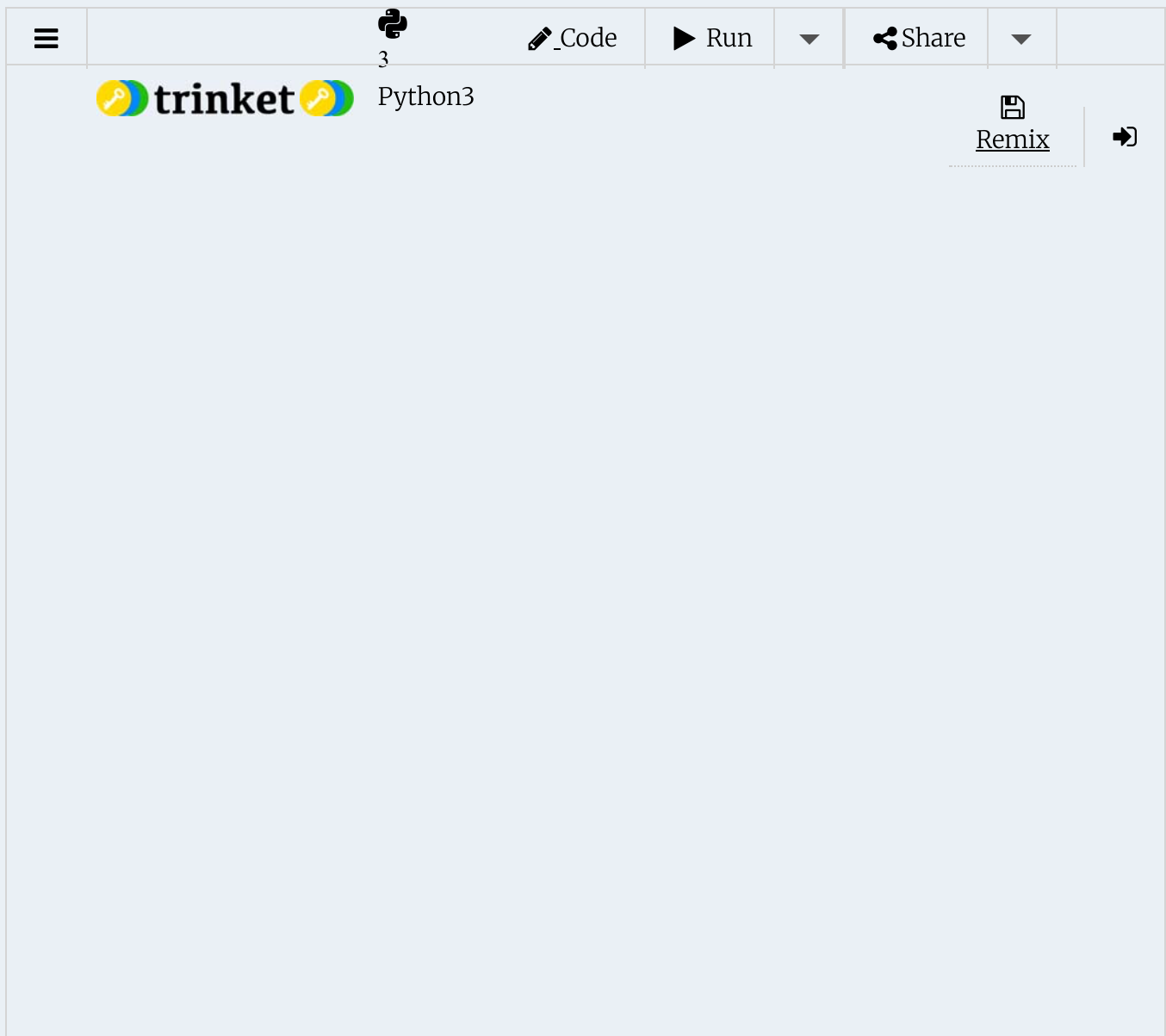
from abc import ABC, abstractmethod
# Abstract base class representing a Shape
class Shape(ABC):
    def __init__(self, colour):
        self.colour = colour
    @abstractmethod
    def calculate_area(self):
        """Abstract method to calculate area (must be implemented by subclasses)"""
        pass
    def get_colour(self):
        """Concrete method to get the colour"""
        return self.colour
# Concrete subclass Circle that extends Shape and implements calculate_area()
class Circle(Shape):
    def __init__(self, colour, radius):
        super().__init__(colour)
        self.radius = radius
    def calculate_area(self):
        return 3.14159 * self.radius * self.radius
# Another concrete subclass Rectangle
class Rectangle(Shape):
    def __init__(self, colour, length, width):
        super().__init__(colour)
        self.length = length
        self.width = width
    def calculate_area(self):
        return self.length * self.width
# Example usage
# my_shape = Shape("Red") # This would raise a TypeError as Shape is an abstract class
my_circle = Circle("Blue", 5)
my_rectangle = Rectangle("Green", 4, 6)
print(f"Circle colour: {my_circle.get_colour()}")
print(f"Circle area: {my_circle.calculate_area()}")
print(f"Rectangle colour: {my_rectangle.get_colour()}")
print(f"Rectangle area: {my_rectangle.calculate_area()}")

```

In this example, `Shape` is an Abstract Base Class with an abstract method `calculate_area()`. Both `Circle` and `Rectangle` are concrete subclasses that inherit from `Shape` and provide their own specific implementations for `calculate_area()`. The `Shape` ABC defines a common interface (the `calculate_area()` method) that all its concrete subclasses must adhere to, thus achieving abstraction.

Try it out

Try creating some more shapes and printing out their states.



Implicit Interfaces and Duck Typing

Python also uses "duck typing," which means the type or class of an object is less important than the methods it supports. If an object "walks like a duck and quacks like a duck," then it's treated as a duck. This allows for a more flexible form of abstraction where classes might not explicitly inherit

from a common abstract base class but still implicitly adhere to a particular interface (set of methods).

Abstraction in Python helps create modular and flexible code by focusing on what objects can do rather than strictly what they are. Using ABCs provides a more formal way to define and enforce these abstract interfaces when needed, promoting better code organisation and maintainability.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**